Control Statements in C

2.3 Nested if else Statement

- Nested if else statements are used to execute different piece of code when we have more than two options to handle.

  Example

  Grade selection based on score.

  Here, if our score is more than 90, we will get S grade

  If our score is in between 80 and 90, we will get A grade

  If our score is in between 70 and 80, we will get B grade

  If our score is in between 60 and 70, we will get C grade

  If our score is in between 51 and 60, we will get D grade

  If our score is 50, we will get E grade

  Below 50 will be declared as U.

  Note: Here, we have more than 2 options to handle.

Syntax

```
if(condition1)
{
 //code to be executed if condition1 is true
}
else if(condition2)
{
 //code to be executed if condition2 is true
}
else if(condition3)
{
 //code to be executed if condition3 is true
}
else
{
 //code to be executed if all the conditions are false
}
```

- It will check each condition sequentially and once the condition succeeds it goes to the corresponding block.
- If none of them are true, then else code part will take place.

Example: Get a number from user and check whether it is positive or zero or a negative number.

```c
#include<stdio.h>

int main()
{
    int num;
    scanf("%d",&num);
    if(num > 0)
    {
        printf("Positive\n");
    }
    else if(num == 0)
    {
        printf("Zero\n");
    }
    else
    {
        printf("Negative\n");
    }
    return 0;
}
```

Sample Program

- Get score from the user and display the grade based on above logic.

```c
#include<stdio.h>
int main()
{
    int score;
    scanf("%d",&score);

    if(score <=100 && score >= 90)
        printf("S");
    else if(score < 90 && score >= 80)
        printf("A");
    else if(score < 80 && score >= 70)
        printf("B");
    else if(score < 70 && score >= 60)
        printf("C");
    else if(score < 60 && score > 50)
        printf("D");
    else if(score == 50)
        printf("E");
    else if(score < 50 && score >= 0)
        printf("U");
    else
        printf("Enter a valid score between 0 and 100");

    return 0;

}
```

Loop Statements in C

Looping Statements

- Looping statements allows us to repeat statements execution till the given condition becomes false.
- It is also called **iteration statements**.

Why Loops?

- Let's assume a situation where we need to print "Good Morning" 10 times.

```
printf("Good Morning\n");
printf("Good Morning\n");
printf("Good Morning\n");
printf("Good Morning\n");
printf("Good Morning\n");
printf("Good Morning\n");
printf("Good Morning\n");
printf("Good Morning\n");
printf("Good Morning\n");
printf("Good Morning\n");
```

What will be the solution?

Is it printing Good Morning 10 times using printf? like this,

No. It's not a good programming practice.

C language provides **looping statements** to handle these kinds of situation.

Looping Statements

- Looping statements allow a set of instructions to be executed repeatedly based on some condition.
- The conditions are set with relational expressions in which a variable is involved and this variable is called the **loop control variable**. There are three expressions associated with every iteration constructs. These expressions contain the loop control variable as one of its operands.
- **Initialization Expression:** This expression initializes the **loop control variable** with a starting value before the loop begins. It is usually defined at the start of the loop.
- **Test Expression:** This is a **relational or logical expression** that checks the condition involving the loop control variable. As long as this condition is true, the loop will continue executing. Once the condition evaluates to false, the loop stops.
- **Update Expression:** This modifies the loop control variable after each iteration of the loop, ensuring that the loop progresses toward its termination. Common updates include increments or decrements of the control variable.

Contd…

Types of Loops in C:

1. **while loop**
2. **do-while loop**
3. **for loop**

Loops can be classified into two types based on when the condition is checked:

1. **Entry-controlled loop**: The condition is checked before the loop starts. If the condition is true, the loop body runs. The loop keeps running as long as the condition is true. Examples include:
   - **while** loop: Checks the condition first, then runs the loop body if the condition is true.
   - **for** loop: Checks the condition first and runs the loop body if the condition is true. It's commonly used for loops with a known number of iterations.
2. **Exit-controlled loop**: The loop body runs first, and then the condition is checked. If the condition is true, the loop runs again. If it’s false, the loop stops. An example is:
   - **do-while** loop:Runs the loop body first, then checks the condition afterward.

while Loop

- The **while** loop is an **entry-controlled loop.**
- The three **loop control elements** are scattered at different places.
- The **initialization expression** appears before the while statement, the **test expression** is placed along with **while** keyword and the **update expression** is within the loop-body.
- Statements inside while loop will repeat till the condition becomes true.

Syntax:

```
initialization expression
while(condition)
{
 //statements
 update expression
}
```

```
Step 1. It will check the condition
Step 2. if the condition is true
Step 3. Statements will execute.
Step 4. And then increment or decrement the value
of variable
Step 5. Again, it will go to step 2
Step 6.else
Step 7.Execution will come out of while loop
```

while Loop

```
initialization expression
while(condition)
{
 //statements
 update expression
}
```

```
int i = 0;

while (i < 5)
 {
   printf("%d\n", i);
   i++;
 }
```

while Loop Example

Example 1: Printing the “Good Morning” 10 times

```
#include<stdio.h>

int main()
{
   int i = 0; //initialization expression

   while(i < 10) // test expression
    {
      printf(" Good Morning\n");
      i++; // update expression
    }

    return 0;
}
```

while Loop Example 2. Get input from the user and print it until the input is 100.

```
#include<stdio.h>

int main()
{
    int num; //initialization expression
    printf("Enter the number\n");
    scanf("%d",&num);
    printf("The enter number is : %d\n",num);

    while(num != 100) //test expression
    {
        printf("Enter the number\n");
        scanf("%d",&num);     //update expression
        printf("The enter number is : %d\n",num); //Printing the number

    } //if the input is 100, loop will terminate

    return 0;
}
```

Example 3. **This program reads a number from the user, divides it by 2 repeatedly, and counts the number of divisions.**

```c
#include<stdio.h>

int main()
{
  int num,count = 0;
  printf("Enter the number\n");
  scanf("%d",&num);

  while(num > 1)
  {
    count++; //increment count variable

    num = num / 2;

  }

  printf("%d\n",count);

  return 0;
}
```

num	=	20
count	=	0

num	num > 1	count++	num = num>>1
20	true	1	10
10	true	2	5
5	true	3	2
2	true	4	1
1	false	End of while loop becuase 1 > 1 is false	

count	=	4

while Loop Example

Example 4: Printing a Table of square of a number upto 10

```
#include<stdio.h>

int main()
{
   int i = 1; //initialization expression

   // while loop to iterate from 1 to 10

    while(i<=10) //test expression
    {
      printf("Number: %d, Square: %d\n", i, i*i);
      i++; // update expression
    }

    return 0;
}
```

while Loop Example

Example 4: Printing a Table of square of a number upto 10

```
#include<stdio.h>

int main()
{
   int i = 1; //initialization expression

   // while loop to iterate from 1 to 10

    while(i<=10) // test expression
    {
      printf("Number: %d, Square: %d\n", i, i*i);
      i++; // update expression
    }

    return 0;
}
```

Output:

Number: 1, Square: 1
Number: 2, Square: 4
Number: 3, Square: 9
Number: 4, Square: 16
Number: 5, Square: 25
Number: 6, Square: 36
Number: 7, Square: 49
Number: 8, Square: 64
Number: 9, Square: 81
Number: 10, Square: 100

while Loop Example

Example 5: Sum of series of numbers

```
#include<stdio.h>
int main()
{
   int n = 1, sum= 0; //initialization expression

   printf("This program sum a series of numbers.\n");
   printf("Enter integers (0 to terminate): ");
   scanf("%d",&n);
   while(n!=0) //test expression
    {
      sum +=n;
      printf("Enter integers (0 to terminate): ");
      scanf("%d", &n);// update expression
    }
   printf(" The sum is: %d\n", sum);

    return 0;
}
```

do while Loop

- We have to use **do while** loop when we need to execute a group of statements at least once.
- Only difference between while and do while loop is that **while** loop will execute statements only when given condition is true whereas **do while** will execute statements **at least once** without checking the condition.
- The do-while loop is an **exit-controlled loop.**
- The three **loop control elements** are placed similar to that in **while** loop.
- The **initialization expression** appears before the while statement, the **test expression** is placed along with while keyword and the **update expression** is within the loop-body.

Example:

Get integer input from user and print it until the input is 100.

Here, we should get at least one input from the user. So, that only we can check whether it is 100 or not. These kind of situation we should go for do while loop.

Contd

Syntax:

```
do
{
    //statements
    update expression
}while(condition);
```

1. It will execute the statements
2. And then checks for condition
3. If condition is true
4. Execution will goto step 1 again
5. else
6. Execution will come out of the do while loop

do-while Loop

Get input from the user and print it until the input is 100.

```
#include<stdio.h>

int main()
{
    int num;

    do
    {   printf("Enter the number (100 to terminate): ");
        scanf("%d",&num);     //getting input from user
        printf("The entered number is %d\n",num); //Printing the number

    }while(num != 100); //if the input is 100, loop will terminate

    return 0;
}
```

for Loop

- Loops are used in programming to repeat the execution of a specific block of code until the given condition becomes false.
- `for` loop is an entry-controlled loop. All its loop control elements are gathered at one place.
- In general, we should use `for` loop when we know the exact loop count.

Example:

1. Print "Good Morning" 10 times

Here we exactly know the loop iteration count which is 10.

2. Print numbers from 1 to 100

Here also we know the iteration count exactly which is 100.

Syntax:

```
for (initialization expression; test condition; update expression)
{
      //body of the loop
}
```

How `for` loop works?

1. **It will initialize the variable.**

   The initialization statement executed only once.

2. **It will check whether the given condition is true or not.**

   if the condition is true,

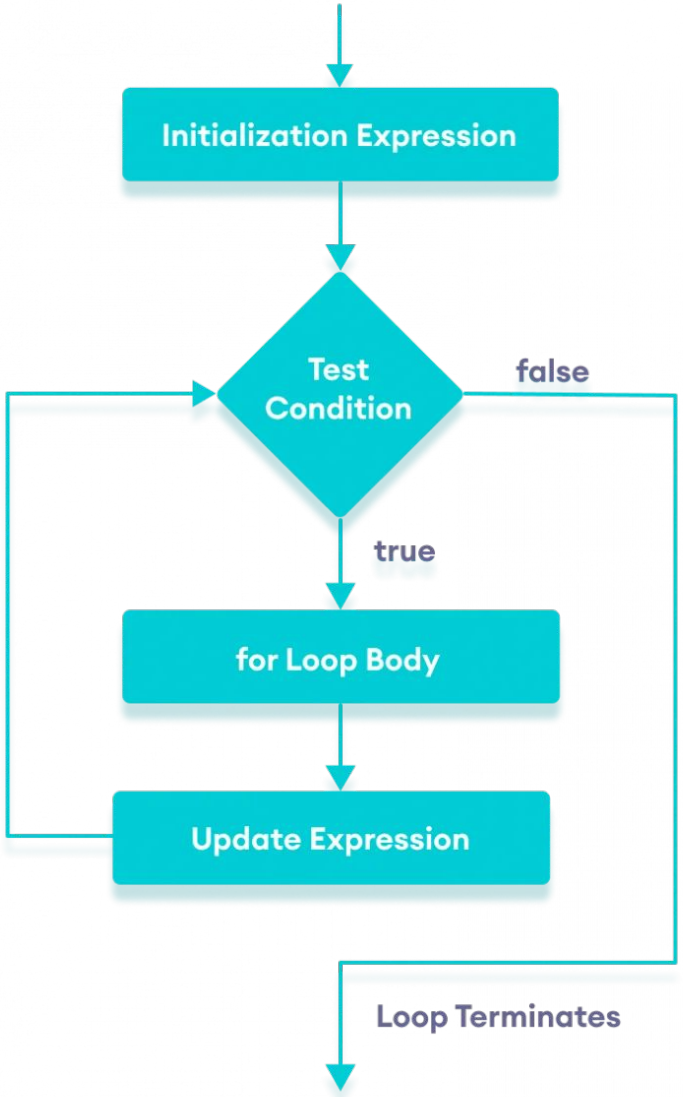
   The statements under the for loop will be executed.

   After that, the value of the variable will be incremented or decremented.

   Again it will follow the above step(2) with the updated variable value.

   else

   The program execution comes out of the "for loop."



for Loop Example

Example1: Printing “Good Morning” 5 times.

```
#include<stdio.h>

int main()
{
    int i;

    for(i=1;i<=5;i++)
    {
        printf("Good Morning\n");
    }

    return 0;
}
```

i	i <= 5	printf("Good Morning \n ");	i++
1	true	Good Morning	2
2	true	Good Morning	3
3	true	Good Morning	4
4	true	Good Morning	5
5	true	Good Morning	6
6	false	End of for loop because 6 <= 5 is false	

for Loop Example

Example2: Print numbers from 1 to 10.

```
#include<stdio.h>

int main()
{
    int i;

    for(i=1;i<=10;i++)
    {
        printf("%d\n",i);
    }

    return 0;
}
```

i	i <= 10	printf("%d\n",i);	i++
1	true	1	2
2	true	2	3
3	true	3	4
4	true	4	5
5	true	5	6
6	true	6	7
7	true	7	8
8	true	8	9
9	true	9	10
10	true	10	11
11	false	End of for loop because 11 <= 10 is false	

for Loop Example

Example 3: Printing a Table of square of a number upto 10

```
#include<stdio.h>

int main()
{
   int i ;

   for(i=1; i<=10; i++)
    {
      printf("Number: %d, Square: %d\n", i, i*i);

    }

    return 0;
}
```